

AI Document Chatbot (Hybrid RAG System)

Project Report

1. Project Overview

The AI Document Chatbot is a full-stack Retrieval-Augmented Generation (RAG) application that enables users to upload and query multiple document formats including PDF, DOCX, XLSX, and PPTX using natural language. The system implements a Hybrid RAG pipeline consisting of semantic chunking, vector search, BM25 keyword retrieval, reranking, and context compression to generate accurate, context-aware responses with citations. The application also supports document management, scoped querying, metadata filtering, and source attribution.

2. Objectives

- Build an intelligent document question-answering system.
- Support ingestion of multiple document formats.
- Implement Hybrid RAG using semantic and keyword retrieval.
- Provide accurate responses with source citations.
- Enable document-scoped querying.
- Develop a user-friendly full-stack interface.

3. Technologies Used

Frontend: React.js, Vite, Tailwind CSS, Axios

Backend: FastAPI, Python, Pydantic

RAG: LangChain, ChromaDB, BM25, CrossEncoder

Embeddings: HuggingFace Embeddings (all-MiniLM-L6-v2)

LLM: OpenRouter API

Other: SemanticChunker, JWT (optional), CORS Middleware

4. System Architecture

React Frontend → FastAPI Backend → Parser Service → Semantic Chunking → Embeddings → ChromaDB + BM25 → Hybrid Retrieval → Reranking → Context Compression → LLM

5. Modules and Features

Document Management Module:

- Upload PDF, DOCX, XLSX, PPTX files
- View uploaded documents
- Delete documents
- Search and filter documents

Chat Module:

- Natural language document querying
- Context-aware responses
- Source citations

RAG Module:

- Semantic chunking
- Vector search using ChromaDB
- BM25 keyword retrieval
- Hybrid search and reranking
- Context compression

Scoped Query Module:

- Query selected documents only
- Metadata-based filtering

6. RAG Pipeline Workflow

1. User uploads document.
2. Parser extracts text from supported file types.
3. SemanticChunker generates meaningful chunks.
4. Metadata is attached to every chunk.
5. Embeddings are generated using HuggingFace models.
6. Chunks are stored in ChromaDB.
7. BM25 index is created for keyword retrieval.
8. Hybrid retrieval combines vector and keyword search.
9. CrossEncoder reranks retrieved chunks.
10. Context compression selects most relevant sentences.
11. LLM generates final answer with citations.

7. Metadata Design

Metadata fields:

- source
- file_type
- chunk_id
- uploaded_at
- page/sheet/slide information

Metadata enables scoped querying, filtering, citation generation, and lifecycle management.

8. API Endpoints

POST /upload

GET /documents

DELETE /documents/{filename}

POST /chat

9. Challenges Faced

- Parsing multiple document formats.
- Managing hybrid retrieval architecture.
- Implementing scoped querying.
- Handling document deletion and vector cleanup.
- Managing BM25 persistence and synchronization.
- Designing efficient metadata filtering.

10. Solutions Implemented

- Developed parser services for multiple file formats.
- Used SemanticChunker for meaningful chunk generation.
- Combined ChromaDB and BM25 for hybrid retrieval.
- Implemented CrossEncoder reranking.
- Added metadata-based scoped querying.
- Implemented document lifecycle management.

12. Future Scope

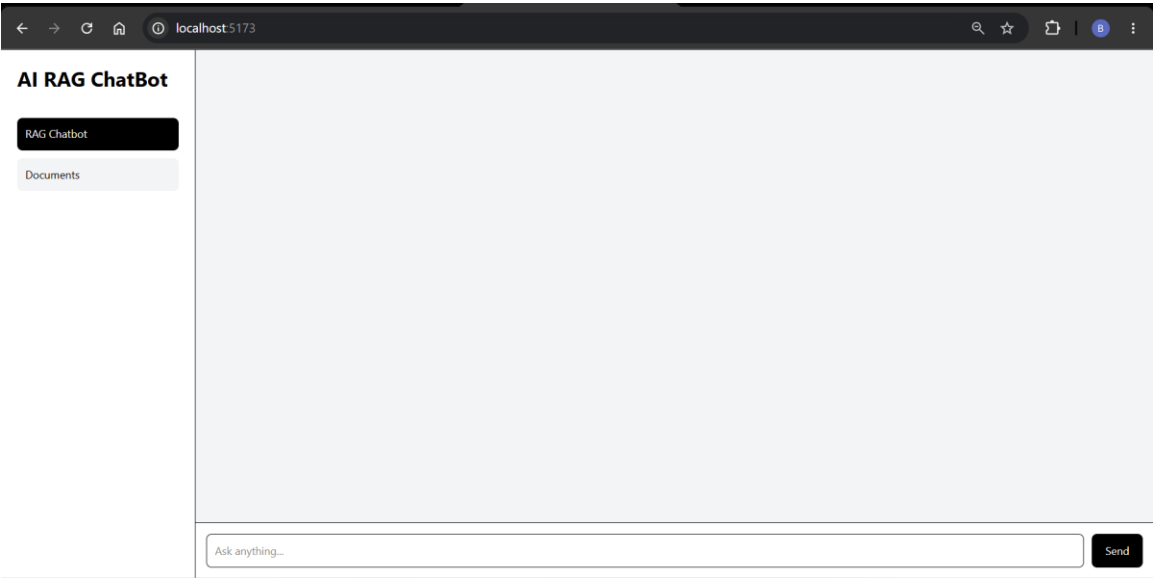
- OCR support for scanned documents.
- User authentication and multi-user support.
- Chat history and conversation memory.
- Cloud deployment and scalability.
- Web search integration.
- LangGraph-based agent workflows.

13. Conclusion

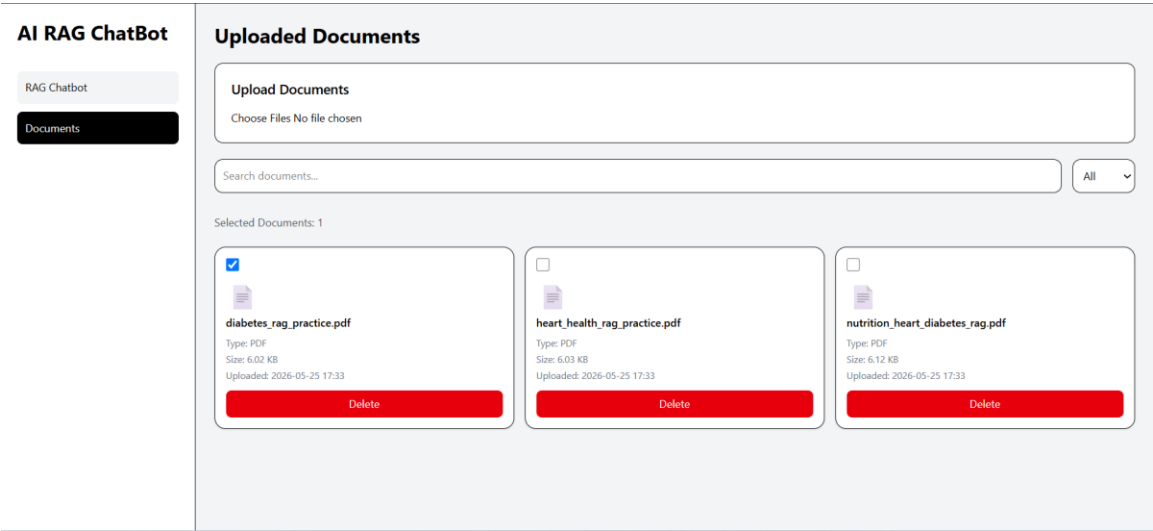
The AI Document Chatbot demonstrates advanced Generative AI and full-stack engineering concepts including Hybrid RAG, semantic search, reranking, context compression, metadata filtering, and document management. The project showcases industry-level AI application development practices.

11. Screenshots

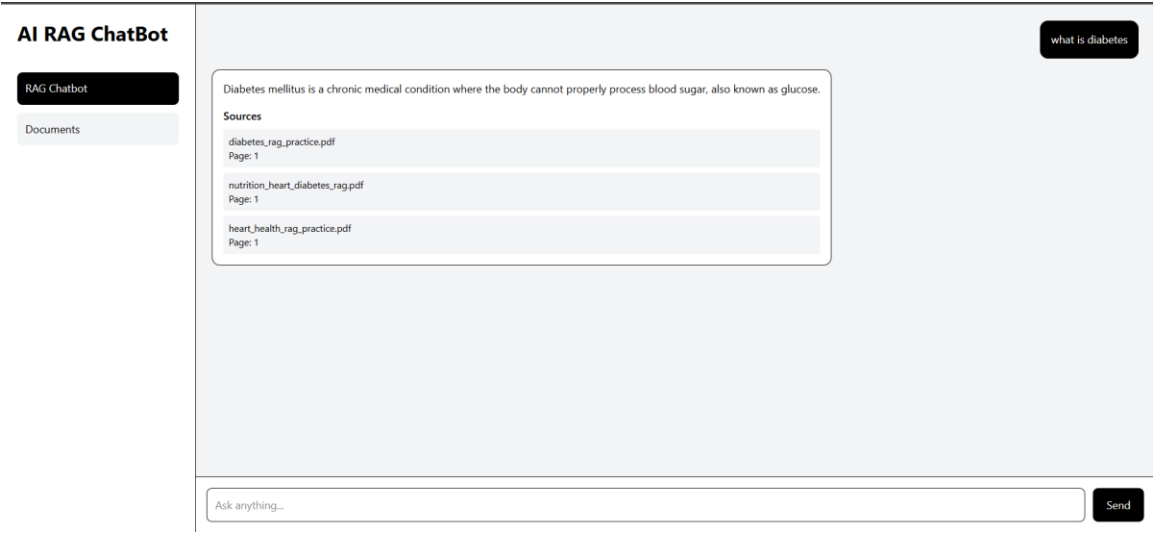
Screenshot 1: _____



Screenshot 2: _____



Screenshot 3: _____



Screenshot 4: _____

